Лабораторна робота № 2

Виконала: Кісельгова Ксенія ТТП-31

Досліджувана програма

Програма на C++ послідовно виконує 100 млн обчислень синуса та 100 млн обчислень квадратного кореня, підсумовує результати та вимірює час виконання за допомогою std::chrono::steady_clock. Атрибут noinline зберігає окремі функції для профілювання. Змінна volatile обмежує деякі оптимізації, що важливо врахувати в аналізі.

```
#include <iostream>
#include <cmath>
#include <chrono>

constexpr int N = 100000000;

__attribute__((noinline))
double calculateSin() {
    volatile double result = 0;
    for (int i = 1; i <= N; ++i) {
        result = result + std::sin(i * 0.001);
    }
    return result;
}

__attribute__((noinline))
double calculateSqrt() {
    volatile double result = 0;
    for (int i = 1; i <= N; ++i) {
        result = result + std::sqrt(i);
    }
    return result;
}

__attribute__((noinline))
double processData() {
    double a = calculateSin();
```

```
    double b = calculateSqrt();
    return a + b;
}


int main() {
    auto start = std::chrono::steady_clock::now();
    double result = processData();
    auto end = std::chrono::steady_clock::now();
    std::chrono::duration<double> elapsed = end - start;
    std::cout << "Result: " << result << '\n';
    std::cout << "Time: " << elapsed.count() << " s\n";
    return 0;
}
```

1. Побудова FlameGraph

1.1. Демонстрація процесу побудови

Для програми без оптимізації зібрано профіль виконання з подією cpu-clock; після збільшення кількості ітерацій зафіксовано 137 зразків. Отриманий файл perf.data перетворено на складені стеки викликів, а далі — на SVG-графік.

```
sudo perf record -F 99 -g -e cpu-clock -- ./program
sudo perf script > perf.out
./FlameGraph/stackcollapse-perf.pl perf.out > perf.folded
./FlameGraph/flamegraph.pl perf.folded > flamegraph.svg
```

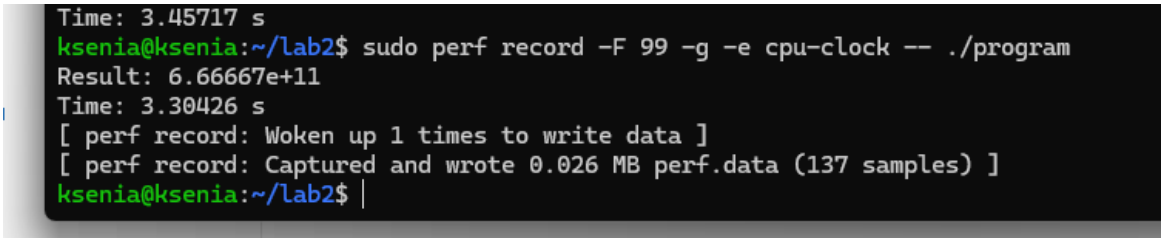


Рис. 1 — запис профілю виконання (137 зразків).

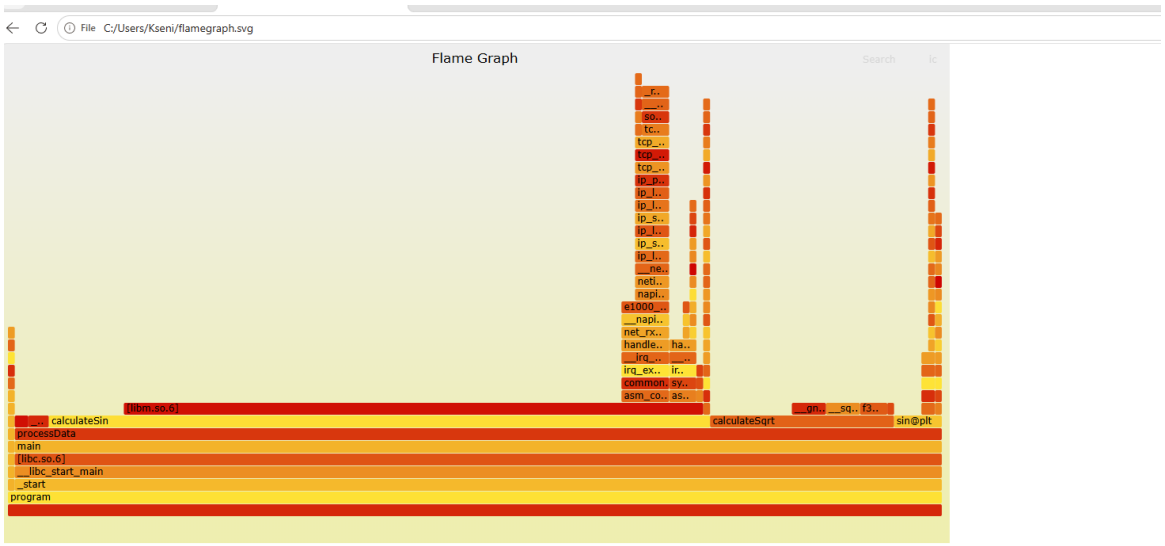


Рис. 2 — FlameGraph для версії -O0.

1.2. Інтерпретація FlameGraph

Основний шлях викликів: program → main → processData → calculateSin / calculateSqrt. Ширина прямокутників відображає частку відібраних стеків, у яких присутня функція; висота — глибину вкладеності викликів, а не тривалість. У профілі домінує гілка calculateSin із викликами математичної бібліотеки libm.so.6. Гілка calculateSqrt помітно менша. Кольори служать переважно для візуального розрізнення блоків, а не відображають швидкість або енергоспоживання. Через обмежену кількість зразків результати мають вибірковий характер.

2. Статистика виконання

2.1. /usr/bin/time --verbose

```
/usr/bin/time --verbose ./program
```

Показник (VirtualBox, -O0)	Значення
Elapsed time	2,87 с
User time	2,43 с
System time	0,44 с
CPU	99 %
Maximum resident set size	4528 КБ
Minor page faults	159
Context switches	2 (1 добровільне, 1 примусове)
Exit status	0

Програма має переважно обчислювальний характер: протягом виконання майже повністю використовувала одне логічне ядро процесора. Максимальне використання пам’яті — близько 4,4 МБ. Код завершення 0 означає успішне виконання.

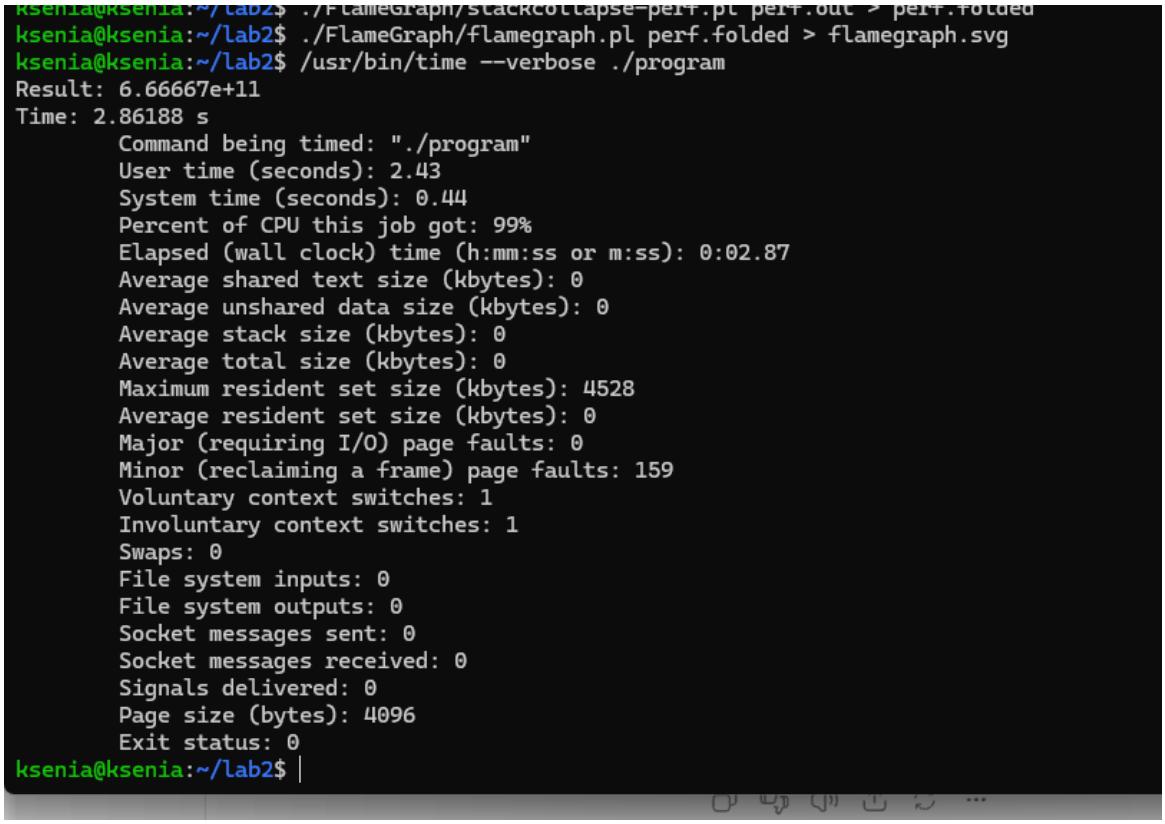


Рис. 3 — результати /usr/bin/time --verbose (-O0).

2.2. perf stat -d

Апаратну статистика для досліджуваної програми; вимірювання -O0 наведена нижче.

```
sudo perf stat -d -- ~/lab2/program_O0
```

Показник (фізичний Linux, -O0)	Значення
Elapsed time	4,170 с
Task clock	4169,15 мс
Cycles	12 021 781 032
Instructions	16 713 067 298
IPC	1,39
Branches	2 354 143 732
Branch misses	576 698 (0,02 %)
L1-dcache-load-misses	376 828

LLC-load-misses	4 953

IPC (instructions per cycle) показує середню кількість інструкцій, виконаних за такт. Показник 1,39 означає близько 1,39 інструкції за такт у цьому запуску. Частота промахів прогнозування переходів невелика, оскільки цикли мають передбачувану структуру. Частина апаратних показників збиралася з мультиплексуванням (відсотки часу роботи лічильників у дужках), тому значення є оцінками perf.

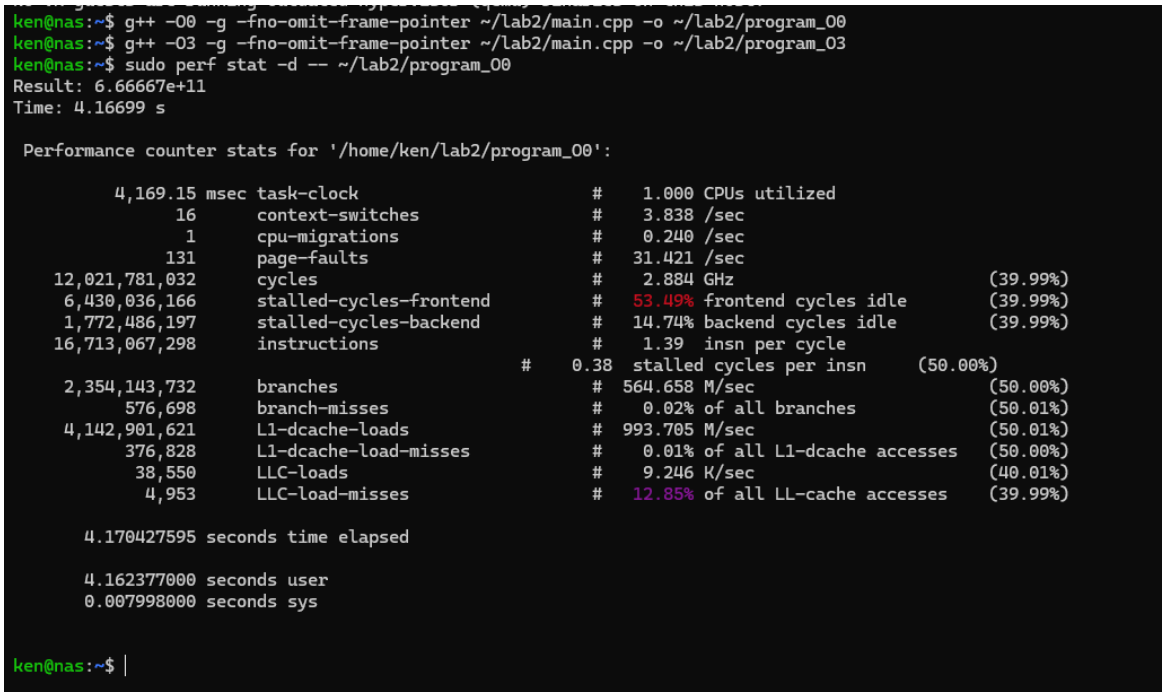


Рис. 4 — perf stat -d для -O0

2.3. perf report (після perf record)

```
sudo perf report --stdio --no-children
```

За результатами вибіркового профілювання в середовищі VirtualBox зібрано 137 зразків, втрачено 0. Серед видимих власних часток (self overhead): calculateSqrt() — 8,76 %, calculateSin() — 8,03 %. Параметр --no-children показує власну вартість функції без витрат її дочірніх викликів, тому ці числа не тотожні ширині гілок FlameGraph. Значну частку витрат calculateSin становить виклик бібліотечної функції sin.

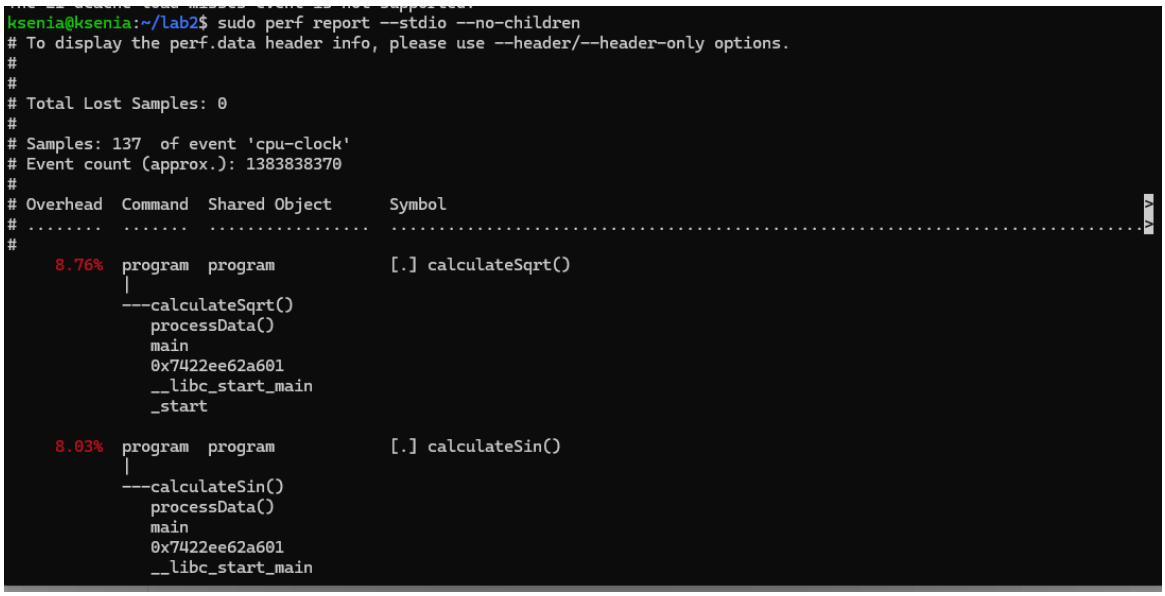


Рис. 5 — частина perf report з власними частками функцій.

3. Вимірювання енергоспоживання

3.1. Енергоспоживання системи

За даними наданого звіту про енергоефективність, зафіксовано базову потужність 21,42 Вт, середню потужність під час роботи версії -O0 — 28,02 Вт, а версії -O3 — 22,02 Вт.

Стан системи	Середня потужність, Вт
Фоновий стан (baseline)	21,42
Виконання програми -O0	28,02
Виконання програми -O3	22,02

Таблиця 1 — Потужність системи за наданим звітом

3.2. Енергоспоживання досліджуваної програми

Для енергетичного розрахунку використано метод оцінки додаткової потужності: $P_{prog} = P_{load} - P_{baseline}$; енергії: $E_{prog} = P_{prog} \times t$. Результат характеризує оцінене додаткове енергоспоживання під час запуску програми, а не ізольовану енергію процесу.

Версія	Час, с	P_load, Вт	P_prog, Вт	E_prog, Дж
-O0	1,58047	28,02	6,60	10,43
-O3	1,30345	22,02	0,60	0,78 (за звітом)

Таблиця 2 — Результати й оцінки енергоспоживання за окремим наданим звітом

4. Порівняння до та після оптимізації

4.1. Порівняння асемблерного коду

g++ -O0 -S -masm=intel main.cpp -o main_O0.s

g++ -O3 -S -masm=intel main.cpp -o main_O3.s

У фрагменті calculateSin для -O0 лічильник циклу зберігається у стеку за адресою [rbp-12], а його значення багаторазово завантажується та змінюється в пам’яті. У версії -O3 лічильник знаходиться у регістрі ebx та змінюється інструкцією add ebx, 1. Компілятор також змінює структуру циклу: порівняння і переходи jle/jne. Виклик sin@PLT зберігається; змінна volatile не дозволяє усунути всі операції читання та запису накопичувача.

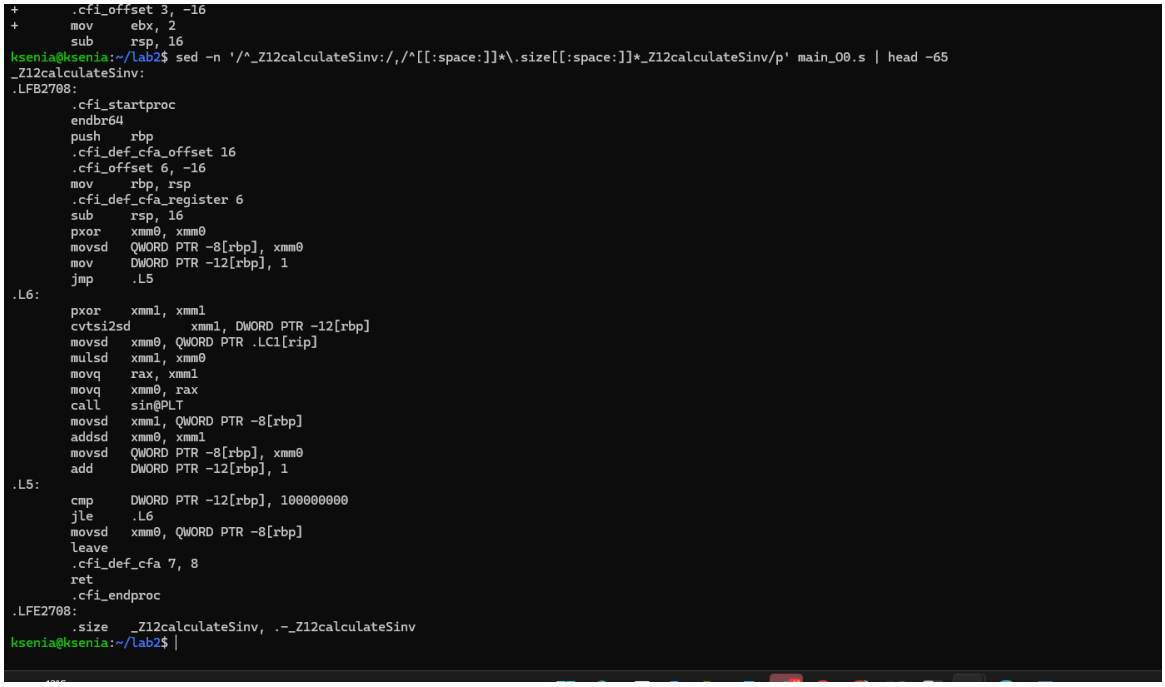


Рис. 6 — фрагмент calculateSin у -O0.

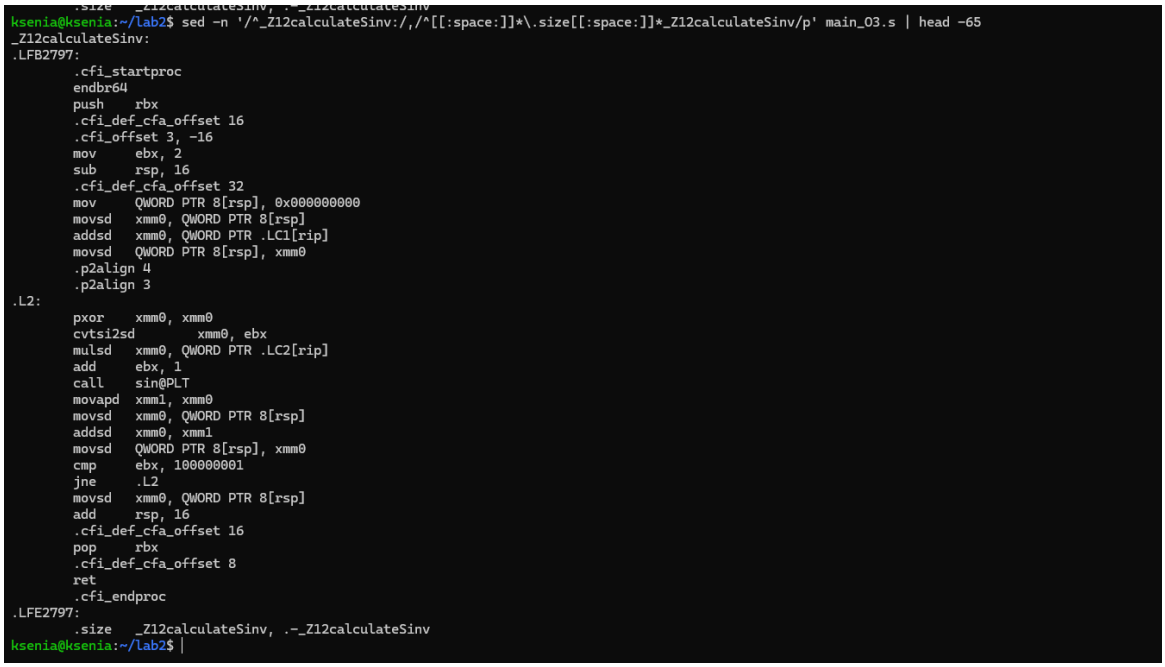


Рис. 7 — фрагмент calculateSin у -O3.

4.1.1. Інструкції AVX/SIMD

Пошук інструкцій роботи з векторними регістрами ymm/zmm і характерних vaddpd/vmulpd/vsqrtpd у main_O3.s збігів не дав. У показаному фрагменті calculateSin використовуються скалярні інструкції на регістрах xmm (зокрема mulsd та addsd), але це не є доказом векторизації кількох елементів одночасно. Виклики sin та volatile обмежують можливості автовекторизації. Тому прискорення не варто приписувати SIMD без додаткового підтвердження.

4.2. Порівняння часу та характеристик виконання

Для зіставності використано вимірювання perf stat -d обох програм.

Показник	-O0	-O3
Час виконання	4,170 с	3,913 с
Cycles	12 021 781 032	11 224 565 073
Instructions	16 713 067 298	14 508 036 928
IPC	1,39	1,29
Branch misses	576 698	581 893
L1 cache misses	376 828	392 566
LLC cache misses	4 953	320

Час скоротився приблизно на 6,2 %, кількість інструкцій — на 13,2 %, тактів — на 6,6 %. IPC дещо знизився, однак загальна кількість інструкцій і тактів стала меншою, тож програма виконалася швидше.

```
ken@nas:~$ sudo perf stat -d -- ~/lab2/program_O3
Result: 6.66667e+11
Time: 3.89188 s

 Performance counter stats for '/home/ken/lab2/program_O3':

          3,894.33 msec task-clock                       #    0.995 CPUs utilized
                14      context-switches                 #    3.595 /sec
                 0      cpu-migrations                   #    0.000 /sec
               133      page-faults                      #   34.152 /sec
    11,224,565,073      cycles                           #    2.882 GHz                         (39.97%)
     6,484,491,878      stalled-cycles-frontend          #   57.77% frontend cycles idle        (40.00%)
     3,011,512,241      stalled-cycles-backend           #   26.83% backend cycles idle         (40.02%)
    14,508,036,928      instructions                     #    1.29  insn per cycle
                                                         #    0.45  stalled cycles per insn     (50.04%)
     1,753,769,379      branches                         #  450.339 M/sec                       (50.06%)
           581,893      branch-misses                    #    0.03% of all branches             (50.04%)
     3,046,229,218      L1-dcache-loads                  #  782.221 M/sec                       (50.01%)
           392,566      L1-dcache-load-misses            #    0.01% of all L1-dcache accesses   (49.99%)
            37,310      LLC-loads                        #    9.581 K/sec                       (39.95%)
               320      LLC-load-misses                  #    0.86% of all LL-cache accesses    (39.95%)

       3.912988802 seconds time elapsed

       3.894451000 seconds user
       0.001000000 seconds sys
```

Рис. 8 — perf stat -d для -O3

Додатково через /usr/bin/time --verbose у VirtualBox отримано 2,87 с (-O0) та 2,53 с (-O3), що відповідає скороченню часу приблизно на 11,8 %. Це окрема серія вимірювань, її не об’єднано з результатами фізичного комп’ютера.

```
Time: 2.96979 s
ksenia@ksenia:~/lab2$ /usr/bin/time --verbose ./program_O3
Result: 6.66667e+11
Time: 2.52138 s
        Command being timed: "./program_O3"
        User time (seconds): 2.34
        System time (seconds): 0.16
        Percent of CPU this job got: 99%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:02.53
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 4552
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 158
        Voluntary context switches: 1
        Involuntary context switches: 35
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
ksenia@ksenia:~/lab2$
```

Рис. 9 — /usr/bin/time --verbose після оптимізації.

4.3. Порівняння FlameGraph

```
sudo perf record -F 999 -g -e cpu-clock -o perf_O3_better.data -- ./program_O3
sudo perf script -i perf_O3_better.data > perf_O3_better.out
./FlameGraph/stackcollapse-perf.pl perf_O3_better.out > perf_O3_better.folded
./FlameGraph/flamegraph.pl perf_O3_better.folded > flamegraph_O3_better.svg
```

У базовому FlameGraph (-O0, 137 зразків) та оптимізованому (-O3, 454 зразки) домінує гілка calculateSin, а calculateSqrt має меншу частку. Оптимізований графік візуально містить менше сторонніх системних стеків, проте це не слід безпосередньо трактувати як зменшення часу системних операцій: збирання відбувалося з різними частотами та різною кількістю зразків. Графіки показують розподіл обчислювальної активності, а кількісне прискорення підтверджують часові вимірювання.

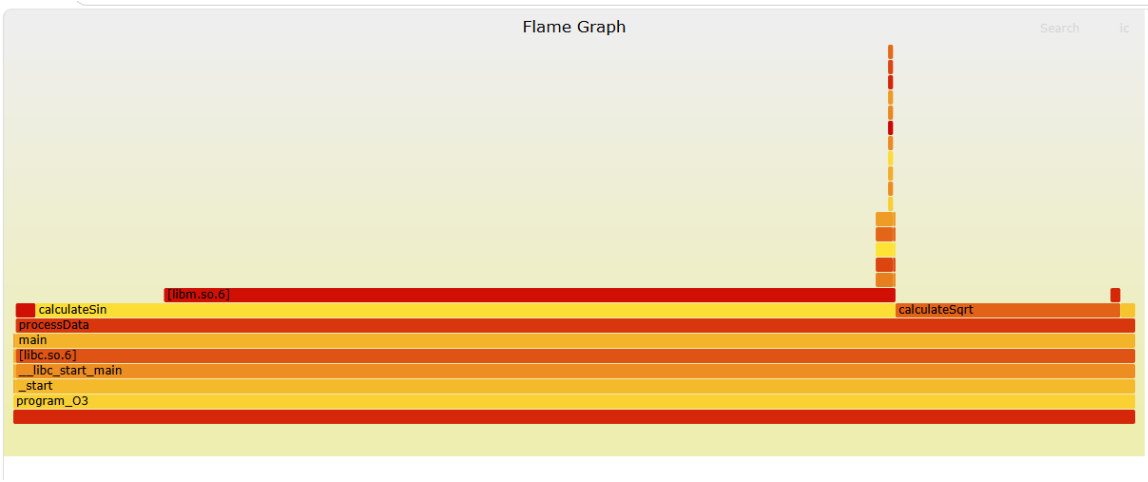


Рис. 10 — FlameGraph програми -O3 (454 зразки).

Для версії -O3 частоту профілювання було збільшено з 99 до 999 Гц, оскільки початкова кількість зразків була недостатньою для детального відображення функцій на FlameGraph.

4.4. Порівняння енергоспоживання

Показник	-O0	-O3	Різниця / зміна
Час виконання, с	1,58047	1,30345	−0,27702 с (−17,5 %)
Потужність під навантаженням, Вт	28,02	22,02	−6,00 Вт (−21,4 %)

Фонова потужність, Вт	21,42	21,42	0
Додаткова потужність P_prog, Вт	6,60	0,60	−6,00 Вт (−90,9 %)
Розрахункова енергія E_prog, Дж	10,43	0,78	−9,65 Дж (−92,5 %)

Таблиця 3 — Порівняння енергоефективності

Аналіз енергоефективності. За наданим звітом версія -O3 виконувалася за 1,30345 с замість 1,58047 с (−17,5 %), потужність під навантаженням становила 22,02 Вт замість 28,02 Вт. Після віднімання постійного фону 21,42 Вт оцінена додаткова потужність зменшується з 6,60 до 0,60 Вт, а розрахункова енергія — з 10,43 до 0,78 Дж.

Висновок

Засобами time, perf і FlameGraph досліджено математичну програму та підтверджено покращення часу виконання після -O3. Додатково звіт містить оцінку додаткової енергії 10,43 Дж (-O0) та 0,78 Дж (-O3), обчислену відносно фонового споживання.